

IO



Quelle

I/O

I/O (Input/Output) bezeichnet die Ein- und Ausgabemechanismen, mit denen Programme Daten lesen (Input) und schreiben (Output) können, z. B. über Tastatur, Dateien, Netzwerke oder Bildschirmausgabe.

Java bietet dafür Klassen im Paket **java.io** und **java.nio**

Blockierend, Nicht-blockierend, Asynchron

Blockierend (typischerweise `java.io`)

beschreibt, wie ein Programmthread vorgeht: Er **stoppt und wartet aktiv**, bis eine bestimmte Aufgabe (wie das Lesen einer Datei oder das Abrufen eines Netzwerk-Ergebnisses) komplett erledigt ist. Solange er wartet, kann dieser Thread keine anderen Aufgaben erledigen.

Nicht-blockierend (typischerweise `java.nio`)

bedeutet, dass der aufrufende Thread **nicht pausiert**, um auf ein Ergebnis zu warten. Stattdessen gibt er die Kontrolle sofort zurück, nachdem die Aufgabe gestartet wurde. Weil das Ergebnis nicht sofort da ist, muss die Operation fast immer asynchron sein; das Resultat wird dir also auf irgendeine Weise später mitgeteilt, d. h., der Aufrufer wartet nie direkt auf das Ergebnis.

Asynchron

bezieht sich darauf, dass eine Operation **im Hintergrund abläuft**, unabhängig vom Hauptprogrammfluss. Wenn eine asynchrone Aufgabe startet, wartet der Aufrufer nicht aktiv auf deren Abschluss. Optional kann auf das Ergebnis gewartet werden (z. B. `get()` oder `join()` bei `CompletableFuture<V>`), was den Thread in diesem spezifischen Moment **blockiert**, bis das Resultat verfügbar ist.

Dateien

**Trennzeichen
zwischen
Verzeichnisse:**
Windows: \
Linux: /

Es gibt zwei Arten von Pfadangaben:

Absolut: C:\finanzen\einkommenssteuererklärung2024.txt

Relativ: ./finanzen/einkommenssteuererklärung2024.txt

Es gibt zwei Arten von Dateitypen:

Zeichenorientiert: z. B. yolo.txt, yolo.json, yolo.xml, yolo.yml, yolo.log

Byteorientiert: z. B. unicorn.jpg, unicorn.mp3, unicorn.mp4, unicorn.zip





Zeichenorientiert

```
{
  "first_name": "John",
  "last_name": "Smith",
  "is_alive": true,
  "age": 27,
  "address": {
    "street_address": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postal_code": "10021-3100"
  },
  "phone_numbers": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "office",
      "number": "646 555-4567"
    }
  ],
  "children": [
    "Catherine",
    "Thomas",
    "Trevor"
  ],
  "spouse": null
}
```

Beispiel:

Die zeichorientierte Datei "john-smith.json", beschreibt eine Person im JSON-Format.

Das JSON-Format beschreibt Daten mithilfe von Key-Value-Pärchen ähnlich wie eine Map.

Der Key und Value wird mit einem Doppelpunkt getrennt. Die Pärchen werden mit Komma getrennt.

Verschachtelungen werden mit {} repräsentiert (quasi eine Map, die als Value wieder eine Map enthält).

Zeichorientierte Daten verbrauchen **mehr Speicherplatz**. Außerdem sind Typumwandlung nötig (z. B. Text zu Integer). Sie sind für Menschen leicht lesbar.

Byteorientiert



```

11111111 11011000 11111111 11100000
00000000 00010000 01001010 01000110
01001001 01000110 00000000 00000001
00000001 00000000 00000000 00000001
00000000 00000001 00000000 00000000
11111111 11011011 00000000 01000011
00000000 00000110 00000100 00000101
00000110 00000101 00000100 00000110
00000110 00000101 00000110 00000111
00000111 00000110 00001000 00001010
00010000 00001010 00001010 00001001
00001001 00001010 00010100 00001110
00001111 00001100 00010000 00010111
00010100 00011000 00011000 00010111
00010100 00010110 00010110 00011010
00011101 00100101 00011111 00011010
00011011 00100011 00011100 00010110
00010110 00100000 00101100 00100000
00100011 00100110 00100111 ...

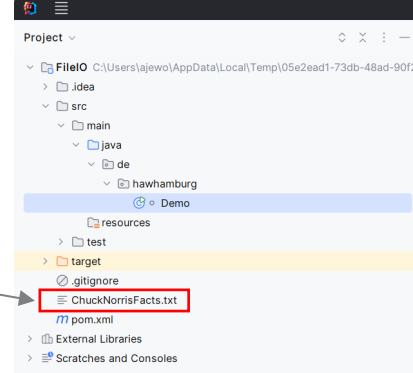
```

[illegible]

Byteorientierte Daten verbrauchen **weniger Speicherplatz**, sind **performanter** und **präziser** (keine Typumwandlung in Text nötig). Sie sind für Menschen aber nicht so leicht lesbar.

java.io.File

Die Datei befindet sich
im Root-Verzeichnis des
Projekts.



Früher wurden in Java Dateien durch die Klasse [java.io.File](#) repräsentiert.
Man findet noch immer viele Code-Beispiele, die java.io.File verwenden.

```
void main() {  
    File file = new File("ChuckNorrisFacts.txt"); // Datei befindet sich direkt im Hauptverzeichnis (Root) des Java-Projekts.  
  
    // AutoClosable reader (and wrapped objects) will automatically be closed after the try-with-resources-block.  
    try (BufferedReader reader = new BufferedReader(new FileReader(file))) { // wrapped character-based reader  
        String line;  
        while ((line = reader.readLine()) != null) {  
            IO.println(line);  
        }  
    } catch (IOException e) {  
        System.err.println("Only Chuck Norris can open this file.");  
    }  
}
```

Dateioperationen werfen in Java
immer eine **Checked IOException**
(oder eine Unterklasse davon)

try-with-resources

Klassen, die das Interface [AutoCloseable](#) implementieren, sollten mit **try-with-resources** verwendet werden (z. B. ExecutorService, BufferedReader, HttpClient).

Java ruft **am Ende** eines **try-with-resources-Blocks** automatisch die **close()**-Methode auf, um die **reservierte Ressource zu schließen** (selbst wenn der try-Block vorzeitig durch **return** oder **throw** beendet wird).

Nicht geschlossene Ressourcen führen bei langlaufenden Programmen zu **Speicherlecks!** 😱

```
void main() {  
    Path filePath = Paths.get("ChuckNorrisFacts.txt");  
  
    // The opened file (reader) will automatically be closed after the try-catch-block.  
    try (BufferedReader reader = Files.newBufferedReader(filePath)) {  
        String line;  
        while ((line = reader.readLine()) != null) {  
            IO.println(line);  
        }  
    } catch (IOException e) {  
        System.err.println("Only Chuck Norris can open this file.");  
    }  
}
```

```
public interface AutoCloseable {  
    void close() throws Exception;  
}
```

Merke: Sobald eine Klasse AutoCloseable implementiert, verwende try-with-resources!

IO-Streams (java.io)

java.io verfügt über viele IO-Streams für unterschiedliche Zwecke oder Daten (siehe Tabelle).

IO-Streams sind **keine** Stream<T>.

IO-Streams sind **AutoClosable** und **Einweg-Straßen**, d. h. **entweder** sie **lesen** oder **schreiben**.

IO-Streams lassen sich **verschachteln**, um sie mit **Methoden** und **Funktionalität** zu **erweitern**.



	Byte-based		Character-based	
	Input	Output	Input	Output
Basic (<i>abstract</i>)	<i>InputStream</i>	<i>OutputStream</i>	<i>Reader</i> , <i>InputStreamReader</i>	<i>Writer</i> , <i>OutputStreamWriter</i>
Arrays	ByteArrayInputStream	ByteArrayOutputStream	CharArrayReader	CharArrayWriter
Files	FileInputStream, RandomAccessFile	FileOutputStream, RandomAccessFile	FileReader	FileWriter
Pipes (Threads)	PipedInputStream	PipedOutputStream	PipedReader	PipedWriter
Buffering	BufferedInputStream	BufferedOutputStream	BufferedReader	BufferedWriter
Filtering	FilterInputStream	FilterOutputStream	FilterReader	FilterWriter
Parsing	PushbackInputStream, StreamTokenizer		PushbackReader, LineNumberReader	
Strings			StringReader	StringWriter
Data	DataInputStream	DataOutputStream		
Data - Formatted		PrintStream		PrintWriter
Objects (Serializable)	ObjectInputStream	ObjectOutputStream		
Utilities	SequenceInputStream			

java.nio.file

Seit JDK 1.7 wurde eine modernere API [java.nio.file](#) eingeführt.

NIO steht für "non-blocking I/O" und bietet mehr Flexibilität, um Dateien zu lesen und zu schreiben, sowie eine "Convenient-API" für einfachen Dateizugriff.

java.nio.file.Path: Repräsentiert einen Dateipfad (Alternative zu java.io.File).

java.nio.file.Paths: Erzeugt Path-Objekte.

java.nio.file.Files: Utility-Klasse, die Methoden zum Erstellen, Kopieren, Löschen, Lesen und Schreiben von Dateien enthält.

java.nio.file.Path



Path repräsentiert einen Dateipfad. `Paths::get` erstellt einen Path.
`Paths::get` und `Path::resolve` kümmern sich automatisch um die korrekte Umwandlung von Dateipfad-Trennzeichen `\` oder `/`.

```
void main() {  
    Path basePath = Paths.get("C:/Users/sara/Documents"); // Path erstellen  
  
    Path relativePath = Paths.get("Projects/X");  
  
    Path fullPath = basePath.resolve(relativePath); // Pfade kombinieren  
  
    IO.println("Base:\t" + basePath);  
    IO.println("Relative:\t" + relativePath);  
    IO.println("Full:\t" + fullPath);  
  
    // Alternativ: Pfadsegmente direkt hinzufügen  
    Path customPath = basePath.resolve("Games").resolve("Diablo");  
    IO.println("Custom:\t" + customPath);  
}
```

Ausgabe:

Base:	C:\Users\sara\Documents
Relative:	Projects\X
Full:	C:\Users\sara\Documents\Projects\X
Custom:	C:\Users\sara\Documents\Games\Diablo

Traversal Path

Achtung:

Durch unsichere Verkettung von Benutzer-Eingaben mit Strings können Angreifer die Spezielle Zeichenkette `..` (bedeutet: gehe ein Verzeichnis hoch bzw. zurück) verwenden, um auf sensible Bereiche des Dateisystems zuzugreifen, die nicht zugänglich sein sollen (**Path-Traversal-Angriff**).

```
void main() {  
    Path home = Paths.get(System.getProperty("user.home")); // Fragt das Betriebssystem nach dem Pfad zum User-Verzeichnis.  
    IO.println(home);
```



```
    Path input = Paths.get("../..secret/passwords.txt");
```

```
    // normalize() replaces . and .. with the actual folders
```

```
    Path path = home.resolve(input).normalize();
```

```
    IO.println(path);
```



```
    if (!path.startsWith(home)) {
```

```
        throw new SecurityException("Path traversal attack detected: " + path);
```

```
    }
```

```
    IO.println(path);
```

```
}
```

Ausgabe:

C:\Users\sara

C:\secret\passwords.txt

java.lang.SecurityException



NIO: zeichenorientiert lesen

In diesem Beispiel wird eine Datei gelesen und NIO kümmert sich darum, dass die Datei automatisch geschlossen wird.

```
void main() {  
    try {  
        Path path = Paths.get("ChuckNorrisFacts.txt");  
        List<String> lines = Files.readAllLines(path); // Ressource wird hier automatisch geschlossen.  
        for (String line : lines) {  
            IO.println(line);  
        }  
        // Files.readAllLines(path).forEach(System.out::println);  
    } catch (IOException e) {  
        System.err.println("Only Chuck Norris can open this file.");  
    }  
}
```

java.nio.Charset

Decimal	Binary	Octal	Hex	ASCII
32	00100000	040	20	SP
33	00100001	041	21	!
34	00100010	042	22	"
35	00100011	043	23	#
36	00100100	044	24	\$
37	00100101	045	25	%
38	00100110	046	26	&
39	00100111	047	27	'
40	00101000	050	28	(
41	00101001	051	29	)
42	00101010	052	2A	*

Dateien bestehen immer aus Bytes (unabhängig davon, ob sie zeichenorientiert oder byteorientiert gelesen oder geschrieben werden).

Ein Charset (Zeichensatz) definiert die Zuordnung von Zeichen zu Bytes und umgekehrt. Beispiele sind [UTF-8](#), ISO-8859-1, Windows-1252, [ASCII](#).

UTF-8 ist plattformunabhängig, unterstützt alle Unicode-Zeichen und ist der de facto Standard für Textdateien.

FYI: Java verwendet intern [UTF-16](#) und konvertiert Zeichen automatisch um.

Unterschiedliche Plattformen und Systeme verwenden unterschiedliche Standardzeichensätze. Ohne die explizite Angabe eines Charsets kann es passieren, dass unterschiedliche Charsets verwendet werden und dadurch Daten falsch interpretiert bzw. unlesbar werden.



NIO: zeichenorientiert lesen mit Stream

In diesem Beispiel kümmert sich NIO nicht darum, die Ressource zu schließen.

"The operating design principle here is '**whoever acquires the resource should release the resource**'. **Files don't auto-close when you read to EOF [End Of File]**; we expect files to be closed explicitly by whoever opened them." [Brian Goetz](#) (Java Language Architect)

```
void main() {  
    Path path = Paths.get("ChuckNorrisFacts.txt");  
    try (Stream<String> lines = Files.lines(path, StandardCharsets.UTF_8)) { // mit optionaler Angabe des Charset.  
        lines.forEach(IO::println);  
    } catch (IOException e) {  
        System.err.println("Only Chuck Norris can open this file.");  
    }  
}
```

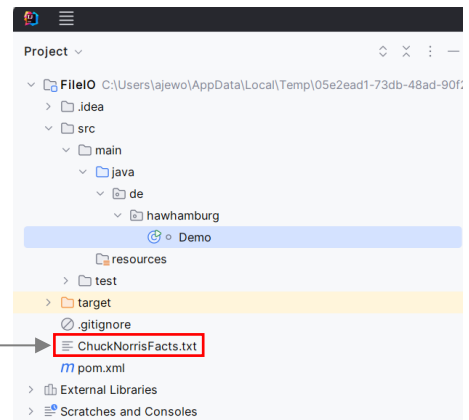


NIO: zeichenorientiert schreiben

In diesem Beispiel kümmert sich die `java.nio.file`-API darum, dass die Datei automatisch geschlossen wird.

```
void main() {  
    List<String> lines = List.of(  
        "Chuck Norris only uses stunt doubles for crying scenes.",  
        "The universe is not expanding; it is running away from Chuck Norris.",  
        "Chuck Norris doesn't use the steering wheel to drive. The road forms where Chuck Norris wants to go.",  
        "When Chuck Norris falls into the water, he doesn't get wet. The water becomes Chuck Norris."  
    );  
    try {  
        Path path = Paths.get("ChuckNorrisFacts.txt");  
        Files.write(path, lines, StandardOpenOption.CREATE, StandardOpenOption.APPEND);  
    } catch (IOException e) {  
        System.err.println("Only Chuck Norris can write this file.");  
    }  
}
```

FYI: `StandardOpenOption.TRUNCATE_EXISTING` würde den vorhandenen Inhalt überschreiben.



Die Datei wird im Root-Verzeichnis des Projekts gespeichert.



NIO: byteorientiert lesen

Bytes werden in NIO mit einem Buffer aus einem FileChannel gelesen.

```
void main() {  
    Path path = Paths.get("ChuckNorrisFacts.txt");  
    Charset charset = StandardCharsets.UTF_8;  
    try (FileChannel channel = FileChannel.open(path, StandardOpenOption.READ)) {  
        ByteBuffer buffer = ByteBuffer.allocate(1024);  
        while (channel.read(buffer) != -1) {  
            buffer.flip();  
            CharBuffer charBuffer = charset.decode(buffer);  
            IO.print(charBuffer);  
            buffer.clear();  
        }  
    } catch (IOException e) {  
        System.err.println("Only Chuck Norris can open this file.");  
    }  
}
```



NIO: byteorientiert schreiben

Bytes werden in NIO mit einem Buffer in einen FileChannel geschrieben.

```
void main() {  
    List<String> lines = List.of(  
        "Chuck Norris only uses stunt doubles for crying scenes.",  
        "The universe is not expanding; it is running away from Chuck Norris.",  
        "Chuck Norris doesn't use the steering wheel to drive. The road forms where Chuck Norris wants to go.",  
        "When Chuck Norris falls into the water, he doesn't get wet. The water becomes Chuck Norris."  
    );  
  
    Path path = Paths.get("ChuckNorrisFacts.txt");  
    Charset charset = StandardCharsets.UTF_8;  
    try (FileChannel channel = FileChannel.open(path, StandardOpenOption.CREATE, StandardOpenOption.APPEND)) {  
        for (String line : lines) {  
            byte[] lineBytes = (line + System.lineSeparator()).getBytes(charset); // System.lineSeparator() liefert den Zeilenumbruch z. B. "\r\n" oder "\n"  
            ByteBuffer buffer = ByteBuffer.wrap(lineBytes); // des derzeitigen Betriebssystems, auf das Java aktuell ausgeführt wird  
            channel.write(buffer);  
        }  
    } catch (IOException e) {  
        System.err.println("Only Chuck Norris can write this file.");  
    }  
}
```